



Vital-ERP

Payroll & HR Management System for Vital Security PLC

The complete "explain it like I'm five" guide:
what it is, how to start it, how to use it, and what problems I found.

Version 1.0 • Generated by Cline from a full code review



Table of Contents

1. **What is this project?** — the simple story
2. **How the system works inside** — the two parts and the brain
3. **The people who use it** — roles explained simply
4. **How to start it on your computer** — every click and command
5. **How to log in** — all the ready-made accounts
6. **How to use every screen** — step-by-step user guide
7. **How a salary gets paid** — the payroll journey, step by step
8. **The API map** — every "door" the system has
9.  **Issues I found** — honest list with severity
10. **Cheatsheet** — quick reference card

1. What is this project? (The simple story)

Imagine a security company in Ethiopia called **Vital Security PLC**.

This company has about **227 guards** who stand guard at banks, shops, and buildings. It also has **office staff** — the people who sit at desks and do paperwork.

Every month, the company must pay all these people. That sounds easy, but it is NOT:

- Guards don't get a fixed salary — they get paid **by the hour**, and hours change every month.
- If a guard works **extra hours** (overtime), they must be paid **1.5× more** per hour.
- If they work on a **holiday**, they get **2× more**.
- Office staff get a **fixed monthly salary**, but if they miss days, money is subtracted.
- The government wants **income tax** and **pension** money taken out (7% from the worker, 11% from the company).
- Some workers have **loans** that must be deducted.
- And very importantly: **no single person is trusted with money**. One person prepares the payroll, another checks it, another approves it, and only then does Finance send it to the bank. This is called **maker-checker**.

Before this app, all of that was done in **Excel files** 🤯. This project replaces the Excel chaos with one website where everyone does their part.

So in one sentence:

💡 One sentence

Vital-ERP is a website that tracks guards and their working hours, calculates everyone's salary automatically (with tax, pension, overtime, and loans), and walks each salary through a chain of approvals before the money is paid.

What the system can do (the big list)

Area	What it does
 Employees	Store every worker's profile: name, code (like VSP-100), category (Guard or Office Staff), bank account, contracts, documents.
 Sites	Every building where guards work is a "site" with an address and a code.
 Guard attendance	Record when guards clock in/out, file hours manually, and see a coverage dashboard ("is every site covered right now?").
 Rotations & shifts	Move guards between sites on a schedule instead of one-by-one.
 Staff attendance	Mark office staff as Present / Absent / Half day / Sick leave, etc.
 Guard payroll	Calculate pay from hours: normal hours + overtime $\times 1.5$ + holiday $\times 2.0$ – tax – pension – loans.
 Staff payroll	Fixed salary minus missing days (each missed day costs salary $\div 22$).
 Approvals	Draft → Checked → Approved → Paid, with "return for correction" if something is wrong.
 Finance	A "Control Tower" page for Finance, plus automatic journal entries (accounting records) when salaries are paid.
 Reports	Payroll summaries and reports by period.
 Audit log	Every important action is written down: who did it, when, from which computer. Like a security camera for data.
 Guard portal	Guards log in and see ONLY their own shift, hours, sites, and salary. They can't see anyone else's.

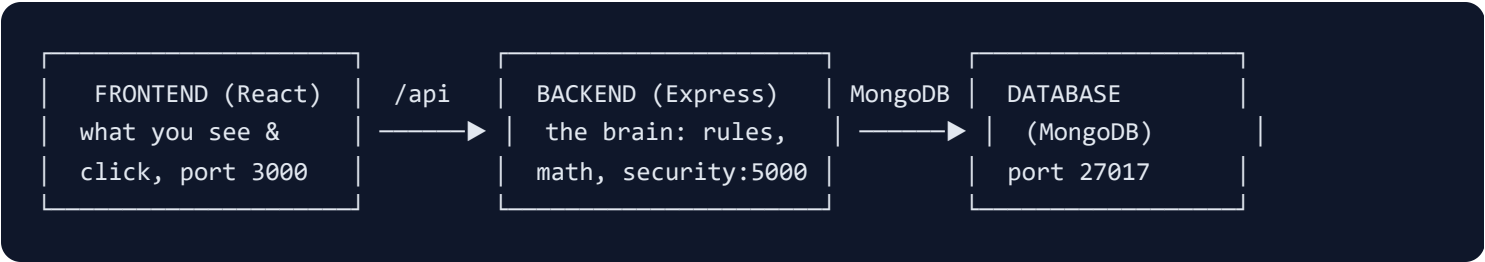
What is NOT finished yet (be aware)

Some menus are just **placeholders** that answer "not yet implemented" (error 501): Inventory, Sales/CRM, Procurement, Manufacturing, Projects, and Fleet. The project is focused on **HR + Payroll** — the other ERP parts are future plans.

2. ⚙️ How the system works inside

Think of the system like a **restaurant**:

- The **Frontend** (React) is the **dining room** — the pretty part you see and click.
- The **Backend** (Express API) is the **kitchen** — it does the real work (calculations, rules, permissions).
- The **Database** (MongoDB) is the **pantry** — where all the information is stored forever.



The frontend at `localhost:3000` secretly forwards every `/api/...` request to the backend at `localhost:5000` (this is configured in `vite.config.ts`).

The backend "brain" has three security layers

Layer	File	What it checks
1. 🔑 Are you logged in?	<code>middleware/auth.ts</code>	Your login "ticket" (JWT token) must be valid. No ticket = rejected.
2. 🛂 Are you allowed?	<code>middleware/rbac.ts</code>	Your role must have the right permission (e.g., only HR can create employees).
3. 📝 Write it down	<code>core/audit/AuditService.ts</code>	Important actions are saved to an audit log: who, what, when, from where.

The main data boxes (database "collections")

Box name	What's inside
User	Login accounts: email, secret password (scrambled with bcrypt), role.
Employee	Every worker: code, names, category (GUARD / OFFICE_STAFF), status, bank info.
GuardProfile	Extra guard-only info: position (Guard or Site Leader), employment type.
Site	Work locations with codes and active/inactive status.
PrimarySiteAssignment	Which guard normally works at which site.
AttendanceRecord	Guard clock-in / clock-out times, and where the data came from.
StaffAttendance	Office staff daily status: Present, Absent, Half day, leaves...
PayrollPeriod	A month and its status: DRAFT → OPEN → CLOSED → LOCKED.
GuardPayrollRecord	One guard's salary for one month: hours, rates, tax, pension, net pay, approval status.
StaffPayrollRecord	Same, but for office staff (fixed salary – missed days).
PayrollApproval	The history of who checked/approved/paid each record.
TaxBracket & PensionRule	The government's tax table and pension percentages (7% / 11%) — changeable in the app, not hardcoded! 👍
SalaryStructure	The "recipe" for pay: earnings, deductions, OT multiplier (1.5), holiday multiplier (2.0).
Loan	Worker loans and the amount to deduct each month.
Contract	Employment contracts with wage, allowances, start/end dates.
JournalEntry	Automatic accounting records created when salaries are paid.
AuditLog	The security camera tape: every sensitive action ever done.

The clever "formula version" idea

👍 Nice design detail

Payroll rules are stored as **versions**

(`PayrollFormulaVersion` and `SalaryStructure`). If the government changes tax rules next year, the admin creates a NEW version with a start date — old months keep their old numbers, so past payslips never change. This is how real payroll systems do it.

3. 👥 The people who use it (roles)

When you log in, the system looks at your **role** and shows you **only the menus you need**. Here are the 8 roles:

Role	Demo login email	What they do (simple)
SUPER_ADMIN	<code>admin@vitalpayroll.com</code>	The boss of the app itself. Can do everything, including payroll configuration (the formula engine).
SYSTEM_ADMIN	<code>sysadmin@vitalpayroll.com</code>	Almost the same as SUPER_ADMIN — technical administrator.
HR_ADMIN	<code>hr@vitalpayroll.com</code>	People-manager. Adds employees, contracts, attendance; starts payroll drafts.
FINANCE_OFFICER	<code>finance@vitalpayroll.com</code>	Money-manager. Reviews payroll, approves, records bank payments.
OPERATIONS	<code>ops@vitalpayroll.com</code>	Site-manager. Manages sites, guards, guard attendance and rotations.
HEAD	<code>head@vitalpayroll.com</code>	Department head — can view payrolls.
CEO	<code>ceo@vitalpayroll.com</code>	Sees everything (read-mostly), final authority.
GUARD	<code>guard@vitalpayroll.com</code>	A normal guard. Gets a special, tiny menu: My Shift, My Hours, My Sites, My Payroll, My Profile. Can't see anyone else's data.

🔑 Password for ALL demo accounts

`password123` — the seed script creates all of them with this same password.

💡 Why so many roles?

It's the

maker-checker

rule: the person who *prepares* the payroll (HR) is different from the person who *checks* it (Finance) and the person who *pays* it (Finance). One person alone can never sneak money out.

4. 🖥️ How to start it on your computer

Step 0 — What you need installed first (one time only)

You need	What it is	How to check if you have it	Where to get it
Node.js (v18 or newer)	The engine that runs JavaScript on your computer	Open PowerShell and type <code>node --version</code> . You should see something like <code>v22.14.0</code> .	nodejs.org — download the "LTS" version and click Next-Next-Next.
MongoDB	The database where everything is stored	Type <code>mongod --version</code> . If "not recognized", it's not installed.	mongodb.com → Community Server. Install as a Windows Service.
Git (optional)	To download/update the code	Type <code>git --version</code> .	git-scm.com

⚠️ Important: MongoDB is NOT installed on this computer!

I checked while making this document: port 27017 (MongoDB's "door") is closed and `mongod` is not found. The app will NOT start until you install MongoDB first. This is also listed in the Issues section.

Step 1 — Create the settings file (one time)

The backend needs a file called `.env` with its settings. A template already exists. In PowerShell:

```
cd C:\Users\dagmaros\Desktop\kal_erp\backend
Copy-Item .env.example .env
```

The file contains:

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/vitalpayroll
JWT_SECRET=your-super-secret-jwt-key-change-in-production
JWT_EXPIRES_IN=24h
NODE_ENV=development
```

👉 For real production use, change `JWT_SECRET` to a long random string! (See Issues, #1.)

Step 2 — Install the "ingredients" (one time)

Each part needs its libraries downloaded. Run these two commands (each takes 1–3 minutes):

```
cd C:\Users\dagmaros\Desktop\kal_erp\backend
npm install

cd C:\Users\dagmaros\Desktop\kal_erp\frontend
npm install
```

Step 3 — Fill the database with starting data (one time)

An empty system is useless — the "seed" script creates demo users, a demo guard, office staff, tax tables, pension rules, salary structures, and the current payroll month:

```
cd C:\Users\dagmaros\Desktop\kal_erp\backend
npm run seed
```

You should see messages like `[SEED] Connected to MongoDB` and `[SEED] Created: admin@vitalpayroll.com (SUPER_ADMIN)`.

Step 4 — Start the backend (every time you work)

```
cd C:\Users\dagmaros\Desktop\kal_erp\backend
npm run dev
```

Wait until you see: `[SERVER] Running on port 5000 in development mode`. **Keep this window open.**

✓ Test it

Open `http://localhost:5000/api/health` in your browser. If you see `{"success":true,"message":"Vital Security API is running"...}` — the kitchen is cooking!

Step 5 — Start the frontend (every time you work, in a SECOND window)

```
cd C:\Users\dagmaros\Desktop\kal_erp\frontend
npm run dev
```

Wait until you see: `Local: http://localhost:3000/`

Step 6 — Open the app! 🎉

Open your browser and go to `http://localhost:3000` → you'll see the login page.

🖥️ Two windows side by side

Backend and frontend each run in their own PowerShell window. Backend = window 1 (`npm run dev` in `backend`), Frontend = window 2 (`npm run dev` in `frontend`). To stop either one, click its window and press `Ctrl + C`.

Making a "production" build (when you want the fast, final version)

```
cd C:\Users\dagmaros\Desktop\kal_erp\backend
npm run build      # compiles TypeScript → dist/
npm start          # runs dist/server.js

cd ../frontend
npm run build      # creates the dist/ website folder
npm run preview    # serves it locally
```

5. How to log in and use every screen

Logging in (the first thing you'll do)

- 1. Open `http://localhost:3000` → the **Login page** appears.
- 2. Type an email from the roles table (e.g., `hr@vitalpayroll.com`) and password `password123` .
- 3. Click **Login**. You land on the **Dashboard** — a page of big colorful cards, one per module you're allowed to use, plus a search box.
- 4. If your session expires (after 24h), the app automatically returns you to the login page.

Magic trick to know

The app looks
different depending on who logs in

. Log in as `ops@` → you see Operations modules. Log in as `guard@` → the whole app shrinks to 5 simple "My ..." pages. Try all the accounts to see it!

The screen map (for non-guard roles)

Module group (color)	Screens inside	Who can see it
 HR & People	Employees, Staff Attendance	SUPER_ADMIN, SYSTEM_ADMIN, HR_ADMIN, CEO
 Operations	Guard Attendance, Sites, Guards, Rotations	SUPER_ADMIN, SYSTEM_ADMIN, OPERATIONS, CEO (+HR/Finance for some screens)
 Payroll	Guard Payroll, Staff Payroll	SUPER_ADMIN, SYSTEM_ADMIN, HR_ADMIN, FINANCE_OFFICER, HEAD, CEO
 Finance	Control Tower, Finance Review, Staff Summary	SUPER_ADMIN, SYSTEM_ADMIN, FINANCE_OFFICER, CEO
 Reports	Reports	Most roles
 Admin	Settings, Salary Structures, Payroll Config	Admins, Finance, CEO (Payroll Config = SUPER_ADMIN & CEO only)

Screen-by-screen guide

Dashboard (/)

Big cards for each module, a greeting ("Good morning..."), your name and role at the top right. Click a card to enter that module. Cards you're not allowed to use are simply not shown.

Employees (/employees) — HR area

1. You see a **table of all employees** with search/filter.
2. Click **"New Employee"** → a form opens: code, names, category (GUARD or OFFICE_STAFF), phone, bank details, etc.
3. Click an employee to **open their profile** → edit them, open their **contract** (wage, allowances, dates), upload **documents** (PDF/photos up to 20 MB), and manage a **guarantor**.

Staff Attendance (/staff-attendance) — HR area

1. Pick a period; you see the office staff list with a status per day.
2. Mark each person: **PRESENT / ABSENT / PAID_LEAVE / UNPAID_LEAVE / SICK_LEAVE / HALF_DAY / HOLIDAY / WEEKEND**.
3. HR "locks" the period when the month is done — **this is required before staff payroll can be generated!**

Sites (/sites) — Operations

1. List of all sites with codes (e.g., a bank branch).
2. **New Site** form: name, code, type (Commercial/Residential/Industrial/Government), address, status.
3. Click a site → **Site Detail**: which guards are assigned there, plus **Site Notes** (digital notes/sign-offs about what happened at the site).

Guards (/guards) — Operations

List of all guards with their profiles (position: Guard or Site Leader, employment type: Permanent/Contract/Temporary, ID card number). Assign a guard's **home site** here.

Guard Attendance (/attendance) — Operations

This is one of the busiest screens. It includes:

- **Guard Attendance Grid** — a calendar-like grid of guards × days where you enter/edit hours.

- **Guard Manual Filing** — enter hours by hand when a guard forgot to clock (with an audit trail of who edited what).
- **Quick Log Panel** — fast clock-in / clock-out for a guard.
- **Coverage Dashboard** — "right now, is every site covered by a guard?" Red = nobody on duty!

Rotations (</rotations>) — Operations

Instead of moving 50 guards one-by-one, create a **rotation plan**: pick guards, pick sites, pick dates — the system swaps them on schedule. Rotations have statuses: DRAFT → ACTIVE → PAUSED → ARCHIVED.

Guard Payroll (</guard-payroll>) — Payroll area

1. **HR/Admin** picks an OPEN payroll period and clicks **Generate** → the system creates one DRAFT record per guard, copying their normal hours, overtime, and holiday hours from attendance.
2. HR can edit hours or enter **regular OT** and **holiday OT** while the record is DRAFT (or RETURNED).
3. The record is **submitted** → Finance **checks** it → it gets **approved**.
4. If something is wrong, Finance clicks **Return for Correction** with a reason → it goes back to HR as RETURNED.
5. After approval, Finance records the **payment** (method + date + reference) → status becomes **PAID** and an automatic **journal entry** is written to the accounting books.

Staff Payroll (</staff-payroll>) — Payroll area

Same idea, but for office staff, and it needs the staff attendance period to be **LOCKED** first. Each record = contract wage – (missed days × daily rate). Workers with no active contract are skipped and listed with a reason.

Finance Review (</guard-payroll/review>) — Finance

Finance's desk: see all submitted payroll records, check the numbers, check them, approve them, or return them with a reason.

Finance Control Tower (</finance>)

Finance's overview: periods, totals, what's waiting for approval, what's paid.

Reports (</reports>)

Payroll summaries by period — who was paid what, totals for the company.

Admin area: manage users, tax brackets, pension rules, payroll rates, and the **formula version** (the recipe engine). Payroll Config is only for SUPER_ADMIN and CEO — handle with care!

👤 **The Guard Portal (log in as `guard@vitalpayroll.com`)**

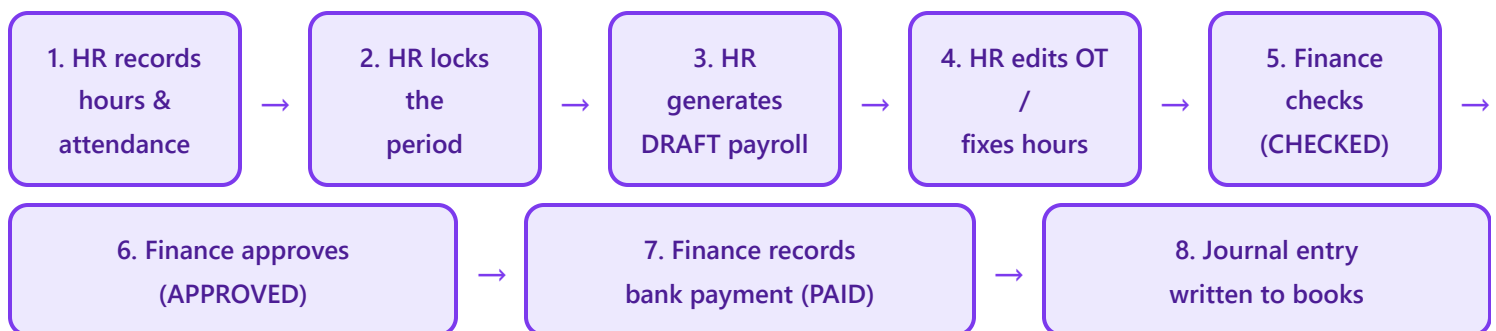
When a GUARD logs in, the app becomes tiny and friendly — 5 pages only:

Page	What the guard sees
🕒 My Shift	Today's shift: which site, what time to be there.
🕒 My Hours	Hours worked this month — normal, overtime, holiday.
📍 My Sites	The site(s) they are assigned to.
💰 My Payroll	Their own payslips: gross, deductions, net pay.
👤 My Profile	Their own info; change their password here.

👉 No matter what address a guard types, they are always redirected to their own pages — they literally cannot reach admin screens.

6. 📁 **How a salary gets paid (the payroll journey)**

Here is the whole life of one payroll month, from start to bank transfer:



The math the computer does for a guard

```
Normal pay    = normal hours × normal rate
Overtime pay  = OT hours × normal rate × 1.5    ← the "OT multiplier"
Holiday pay   = holiday hours × normal rate × 2.0 ← the "holiday multiplier"

GROSS        = Normal pay + Overtime pay + Holiday pay + allowances

Income tax    = looked up in the TaxBracket table (progressive: bigger
                salary → bigger % — the exact % comes from the database)
Employee pension= 7% (percentage from the PensionRule table)
Employer pension= 11% (the COMPANY pays this extra, not taken from the worker)
Loan          = whatever monthly amount is set on the Loan record

NET PAY = GROSS - Income tax - Employee pension - Loan - other deductions
```

The math for office staff

```
Daily rate = monthly wage ÷ 22 working days
Missed money = (ABSENT days + UNPAID_LEAVE days + ½ × HALF_DAY days) × daily rate
Basic salary = wage - missed money
then the same tax / pension / loan deductions as above.
```

Safety rules built into the workflow

- Guard payroll can only be generated while the period is **OPEN**.
.
- Staff payroll refuses to generate unless staff attendance is **LOCKED**
(nice clear error message included).
- Hours/OT can only be edited in **DRAFT**
or **RETURNED**
status — approved records are frozen.
- Every check/approve/pay/return step is saved in `PayrollApproval` + `AuditLog` — you can always answer "who approved this?".

7. 🗺️ The API map (every "door" the system has)

You don't need this section to USE the app — it's for developers and for testing with tools like Postman. All endpoints require a login token except `/api/health` and `/api/auth/login`.

Base path	What it manages
<code>/api/auth</code>	Login, register, "who am I", change password, update profile
<code>/api/users</code>	User accounts (admin)
<code>/api/employees</code>	Employee CRUD
<code>/api/sites</code> , <code>/api/site-notes</code>	Sites and their notes
<code>/api/guards</code>	Guard profiles & assignments
<code>/api/attendance</code>	Guard attendance (clock, manual filing, grid)
<code>/api/staff-attendance</code>	Office staff daily attendance + lock period
<code>/api/shifts</code> , <code>/api/rotations</code>	Shift templates, assignments, rotation plans
<code>/api/guard-payroll</code>	Generate, list, edit hours/OT, submit, check, approve, pay, return
<code>/api/office-payroll</code>	Same workflow for office staff
<code>/api/finance</code>	Finance overview & review endpoints
<code>/api/journal</code>	Accounting journal entries
<code>/api/rules</code>	Tax brackets, pension rules, payroll rates
<code>/api/admin/payroll-config</code>	Payroll formula versions
<code>/api/salary-structures</code>	Salary structure "recipes"
<code>/api/contracts</code> , <code>/api/guarantors</code> , <code>/api/candidates</code> , <code>/api/performance</code>	HR extras: contracts, guarantors, onboarding, performance reviews
<code>/api/departments</code> , <code>/api/positions</code>	Organization structure
<code>/api/reports</code>	Reports
<code>/api/audit-logs</code>	The audit trail (read-only)

<code>/api/files</code>	Document upload/download (max 20 MB; pdf/jpg/png/doc/docx only)
<code>/api/health</code>	Free "are you alive?" ping — no login needed
<code>/api/v2/inventory, /api/v2/sales-crm, /api/v2/procurement, /api/v2/manufacturing, /api/v2/projects, /api/v2/fleet</code>	Stubs — always answer "not yet implemented" (501)

8. Issues I found (honest list)

I read the code and ran the checks myself. Here is everything I found, worst first.  = fix before real money goes through this system,  = should fix soon,  = nice to fix.

Critical

1. The JWT secret has a weak fallback — anyone could forge login tickets

In `backend/src/config/env.ts`, if `JWT_SECRET` is missing in `.env`, the code silently uses `'fallback-secret-change-me'`. Anyone who guesses this well-known string can create valid login tokens for ANY role, including SUPER_ADMIN.

Fix:

fail fast at startup if `JWT_SECRET` is not set, instead of using a fallback.

2. MongoDB is not installed on this computer

I verified: port 27017 is closed and `mongod` is not found. Nothing will run until MongoDB Community Server is installed.

Fix:

install from [mongodb.com](https://www.mongodb.com) and let it run as a Windows Service.

3. Uploaded documents are served with NO login check

`app.ts` line 50: `app.use('/uploads', express.static(...))` — the folder holding employee documents (IDs, contracts, photos) is public. Anyone who knows/guesses a file URL can download it without logging in. (The upload *endpoint* is protected, but the files themselves are not.)

Fix:

serve uploads through an authenticated route.

● Important

4. Same password for every demo account + it's in the source code

The seed script gives all 8 accounts `password123`. Fine for demos — dangerous in real use.

Fix:

force password change on first login before going live.

5. Duplicated legacy modules — a maintenance trap

`backend/src/modules/` contains BOTH the `hr/` tree AND older copies of the same modules at the top level (`attendance/` , `finance/` , `guards/` , `guardPayroll/` , `officePayroll/` , `sites/` , `staffAttendance/` , `employees/` ...). Only the `hr/` copies are wired into `app.ts` — the others are dead code that still compiles. A developer could fix a bug in a dead copy and see no effect.

Fix:

delete the unused copies.

6. Duplicate database index warnings

On every startup, Mongoose warns about duplicate indexes on `email` , `employeeCode` , `siteCode` , and `code` — the schemas declare `unique: true` AND a manual `schema.index()` for the same field. Harmless but noisy.

Fix:

remove the redundant `schema.index()` lines (e.g., in `Employee.ts` line 80).

7. Hardcoded business numbers

`GUARD_MONTHLY_HOURS = 720` (payrollCalculation.service.ts) and `dailyRate = salary / 22` (staff payroll) are hard-wired in code. If policy changes, a developer must edit code.

Fix:

move them into Settings/PayrollConfig like the tax and pension rules already are.

8. No automated tests

There are PowerShell smoke-test scripts in the repo root, but no real unit/integration test suite (no Jest/Vitest). Payroll math is exactly the kind of code that needs automated tests — a wrong tax bracket would silently mispay 227 people.

● Minor / housekeeping

9. Junk committed to the repository

Word temporary lock files (`~$tal_Security...docx`), test logs (`*_log.txt` , `tsc-check.txt` , `backend_err.txt`) are committed. Add a `.gitignore` with `~$*` and log patterns and remove them.

10. `tsc-check.txt` errors are already fixed — don't be alarmed

That old file lists TypeScript errors in `EmployeeList.tsx` (missing `selectedEmployee`). I re-ran the type checker on the current code:

both backend and frontend pass cleanly (0 errors)

. The file is just stale history.

11. CORS is wide open

`app.use(cors())` allows requests from any website. Fine for development; restrict it to the real frontend domain in production.

12. README is nearly empty

`README.md` contains only "Vital-ERP / ERP". This document is a good starting point — consider copying the setup section into the README.

13. Frontend production API path

The axios client uses `baseUrl: '/api'` , which only works when something proxies `/api` to the backend (Vite does this in dev). In production you need a reverse proxy (nginx/IIS) or a configurable API URL.

14. No rate limiting on login

Anyone can try passwords forever on `/api/auth/login` . Add express-rate-limit (e.g., max 5 tries per minute) before going live.

15. Deep-clone caveat (this machine)

The repository was cloned with `--depth 1` because of unstable internet, so only the latest commit exists locally. Run `git fetch --unshallow` when the network is stable if you need history.

9. Cheatsheet (print this page!)

Start the system

```
# Terminal 1 – MongoDB must be installed & running first!
cd C:\Users\dagmaros\Desktop\kal_erp\backend
npm run dev                # API on http://localhost:5000

# Terminal 2
cd C:\Users\dagmaros\Desktop\kal_erp\frontend
npm run dev                # App on http://localhost:3000
```

One-time setup (new computer)

```
cd backend; Copy-Item .env.example .env; npm install; npm run seed
cd frontend; npm install
```

Logins (password: `password123`)

<code>admin@vitalpayroll.com</code>	Super Admin — everything incl. Payroll Config
<code>sysadmin@vitalpayroll.com</code>	System Admin
<code>hr@vitalpayroll.com</code>	HR — people, attendance, payroll drafts
<code>finance@vitalpayroll.com</code>	Finance — review, approve, pay
<code>ops@vitalpayroll.com</code>	Operations — sites, guards, attendance, rotations
<code>head@vitalpayroll.com</code>	Head — view payrolls
<code>ceo@vitalpayroll.com</code>	CEO — see everything
<code>guard@vitalpayroll.com</code>	Guard — "My shift / hours / sites / payroll / profile" only

Handy URLs

URL	What
<code>http://localhost:3000</code>	The app (login page)
<code>http://localhost:5000/api/health</code>	Backend health check
<code>mongodb://127.0.0.1:27017/vitalpayroll</code>	Database connection string

Status words you'll see

Word	Meaning
DRAFT	Just created, still editable
SUBMITTED / CHECKED	Sent for review / reviewed by Finance
APPROVED	Approved for payment, frozen
PAID	Money sent, journal entry written
RETURNED	Sent back for correction (with a reason)
OPEN / LOCKED (period)	Month accepting entries / month frozen for payroll